BABEŞ-BOLYAI UNIVERSITY, CLUJ-NAPOCA, ROMÂNIA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Compliance-as-Code AI Auditor

Checking Applications Against Policies and Regulations (e.g., EU AI Act)

– Advanced Machine Learning report –

Seminar 4 version: pipeline description and first results

Authors

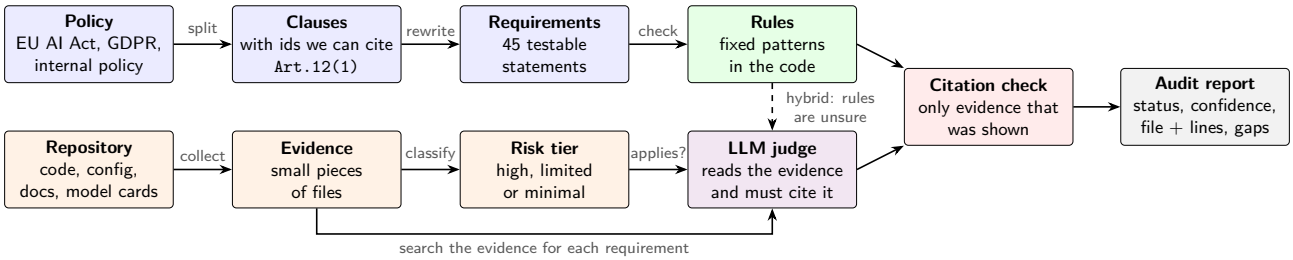
Bâlc Felicia, Sisteme Informatice Avansate, 251, felicia.balc@stud.ubbcluj.ro
Drăghici Eduard, Sisteme Informatice Avansate, 251, eduard.draghici@stud.ubbcluj.ro
Gog Raluca, Sisteme Informatice Avansate, 251, raluca.maria.gog@stud.ubbcluj.ro

2026-2027

Abstract

The EU AI Act [4] and GDPR [3] ask AI systems to keep logs, tell users when they talk to an AI, let a human stop the system, and more. Today, checking an application against these rules is a slow, manual audit. We build a *Compliance-as-Code AI Auditor*. It turns a policy into 45 testable requirements and a code repository into small pieces of evidence. Each requirement is then decided in one of three ways: fixed rules, an LLM judge (Claude Sonnet) that must cite its evidence, or a hybrid that asks the LLM only when the rules are unsure. We test them on five AI applications that we labelled by hand (165 labels). The hybrid is the most accurate (86.1%, against 79.4% for the rules and 71.5% for the LLM) and costs $0.67 instead of $4.81. The LLM alone is the only method that never calls a real gap compliant, and every citation is valid.

Graphical abstract.



1 Project and team

Project: Compliance-as-Code AI Auditor: Checking Applications Against Policies and Regulations (e.g., EU AI Act) (Advanced Machine Learning, FMI, Babeş-Bolyai University, 2026-2027, topic #5). Code: `app/backend` (the auditor), `app/benchmark` (labelled apps and evaluation), `app/notebooks` (data analysis and figures).

Table 1.1: Team members and roles.

Member	Identification	Role
Bâlc Felicia	Sisteme Informatice Avansate, group 251 `felicia.balc@stud.ubbcluj.ro`	*Policy lead:* reads the regulations, writes the requirements and the labelling rules.
Drăghici Eduard	Sisteme Informatice Avansate, group 251 `eduard.draghici@stud.ubbcluj.ro`	*Architecture and LLM lead:* pipeline, decision methods, LLM judge and its prompt.
Gog Raluca	Sisteme Informatice Avansate, group 251 `raluca.maria.gog@stud.ubbcluj.ro`	*Evidence and evaluation lead:* evidence collector, search, benchmark apps, experiments, figures.

Timeline (semester weeks). 1–3 analysis and literature (all) · 2–5 policies and requirements (FB) · 3–7 evidence collector, search, rules (RG, ED, FB) · 5–9 LLM judge (ED), benchmark and labels (FB, RG) · **Seminar 4: first (v0) results, this report** · Seminars 6–7: planned work in Chapter 6.

2 Introduction

2.1 What? Why? How?

What. We take a policy, such as the EU AI Act, GDPR or a company's own AI policy, and the code of an AI application. For every obligation in the policy, we decide whether the application meets it.

Each answer must point to its proof: a file and the lines in it. This is *compliance-as-code*: compliance is checked automatically, like unit tests or Open Policy Agent rules [11].

Why. The AI Act gives different duties to systems with different risk levels, and its text is often vague: it says what to achieve, not how [14]. The proof is spread over code, configuration and documents. A manual audit is out of date after the next commit. Simple text search misses things, and LLMs sometimes invent facts [7].

How. One pipeline prepares the inputs: the policy becomes requirements, the repository becomes evidence, and the risk tier decides which requirements apply. Then three methods decide each requirement: fixed rules, an LLM judge that reads retrieved evidence [8], and a hybrid of the two.

First results. The hybrid is the most accurate (86.1%, against 79.4% for the rules and 71.5% for the LLM) at 14% of the cost of the LLM alone. But it keeps the rules' 12.3% false-compliance rate. The LLM alone finds every gap, but it is too strict.

2.2 Paper structure and original contribution(s)

Our contributions are:

- a simple, precise definition of the task: give each requirement a status, a confidence and the evidence behind it (Chapter 3);
- a pipeline with three interchangeable decision methods and a citation check (Chapter 5);
- a benchmark of five AI applications with 225 labels, including traps for rules and LLMs, and a comparison of the three methods with significance tests (Section 5.2).

Chapter 4 reviews related work, Chapter 5 gives the approach, results and discussion, and Chapter 6 concludes and lists the planned work.

3 Scientific Problem

3.1 Problem definition

Informal description. An auditor must answer one question for each obligation: does this application do what the law asks, and where is the proof? This is hard for three reasons:

- **The law is not code.** Article 12 asks for "automatic recording of events". In code this can be a `logger.info(...)`, a custom `DecisionJournal.record(...)`, or nothing at all, even if the README says it exists.
- **The proof is scattered.** It can be in code, configuration, infrastructure files or documents.
- **Traps are common.** Commented-out code, TODO notes and marketing claims look like proof, but they are not.

Why an intelligent method? Deciding whether some evidence satisfies a vague legal sentence is a reading-and-reasoning task, and LLMs are the best current tool for it. There is no labelled dataset to train a classifier, so we compare two realistic options and their mix. Rules are cheap, repeatable and easy to explain, but brittle. An LLM is flexible, but costs more, can give different answers, and may invent evidence.

Formal definition. The **inputs** are a policy and a repository. The policy is split into clauses, and we rewrite them into testable requirements. The repository is split into pieces of evidence. For each requirement, the **output** is:

- a **status**: compliant, partial, non-compliant, needs review or not applicable;

- a **confidence** between 0 and 1;
- the **cited evidence**: file and lines.

Every answer must follow three rules:

1. a requirement is *not applicable* only if the application's risk tier excludes it;
2. the method may cite only evidence that it was shown;
3. it may never say *compliant* (or partial) without evidence.

"Needs review" means the method does not decide and a human takes over. The worst error is *false compliance*: a real gap that we wrongly call compliant.

Metrics. We score only the applicable requirements. A *gap* is a requirement that is truly partial or non-compliant.

$$\text{accuracy} = \frac{\#\,\text{correct statuses}}{\#\,\text{applicable requirements}}, \qquad \text{false-compliance rate} = \frac{\#\,\text{gaps called compliant}}{\#\,\text{gaps}}. \quad (3.1)$$

We also report macro-F1 (the average F1 over compliant, partial and non-compliant), *gap recall* (the share of gaps we flag) and *gap precision* (the share of our flags that are real gaps), whether the risk tier is right, whether the citations are valid [5], and the cost: LLM calls, dollars and seconds.

4 State of the art/Related work

We found no published system that checks a whole application repository against a regulation on a public benchmark. Related work solves neighbouring problems, grouped below.

4.1 Policy-as-code

Tools such as Open Policy Agent [11] write rules as code and check them on structured files, for example deployment or cloud configurations. Static analysis tools report their findings in a shared format, SARIF [10]. These rules are fast and exact, but they only see text patterns.

Difference. Our rules are linked to legal clauses, and requirements that no pattern can check go to an LLM. On data governance, rules alone reach 35.7% accuracy in our benchmark and the LLM 92.9%.

4.2 Compliance checking of legal documents

Torre et al. [13] and Amaral Cejas et al. [1] use NLP and machine learning to check privacy policies and data processing agreements against GDPR. LegalBench [6] tests how well LLMs reason about legal text.

Difference. We check code, configuration and documents, not only documents, so we can catch a README that promises something the code does not do. We also measure false compliance, the error that matters most for an auditor.

4.3 LLMs as judges, with retrieved evidence

Retrieval-augmented generation lets an LLM answer from passages found by a search engine [8]; we use the classic BM25 search [12]. LLMs are also used as judges, but they have known biases [15].

They can state things that the sources do not support [7], so newer work checks whether each claim is backed by a cited source [5].

Difference. Our judge may cite only the evidence it was shown, and a simple program checks this on every answer, for free. In v0, 100% of the citations are valid.

4.4 AI Act and AI auditing

Veale and Zuiderveen Borgesius [14] analyse the AI Act and show where its text is vague. Mökander et al. [9] propose to audit LLM systems at three levels: the company, the model and the application. Both works are written for people who do the audit by hand.

Difference. Our audit is automatic and runs on the application itself, so it can be repeated after every commit.

5 Investigated approach

5.1 Proposed approach - methodology

5.1.1 Pipeline

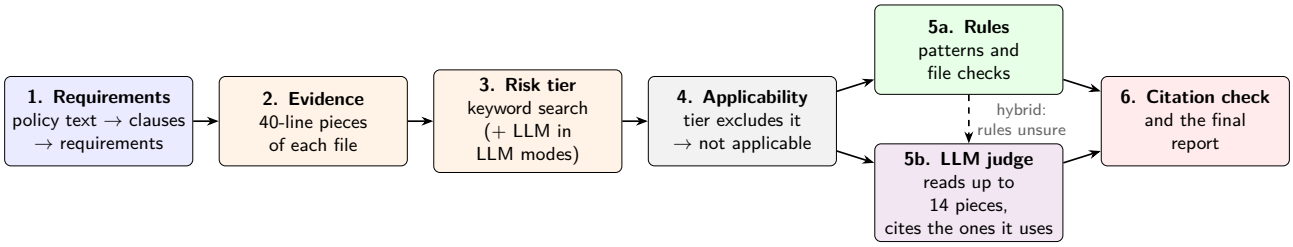


Figure 5.1: The v0 pipeline. The three methods differ only in step 5: rules only (5a), LLM only (5b), or hybrid (5a, then 5b when the rules are unsure).

The auditor is a Python package with six steps (Figure 5.1). (1) The policy text is split into clauses with ids we can cite, and we rewrite them by hand into requirements. (2) The repository is cut into overlapping pieces of 40 lines and indexed for BM25 search [12]. (3) A keyword search (plus one LLM call in the LLM modes) sets the AI Act risk tier. (4) The risk tier decides which requirements apply; no model is involved here. (5) The rules, the LLM judge or the hybrid decide each applicable requirement. (6) A citation check validates every answer, and the result is written as a JSON and a markdown report.

5.1.2 Exploratory data analysis

We need two inputs: the policies (what we check) and the application repositories (what we check it on). With our labels, the repositories are also the test set. Table 5.1 gives the key numbers.

Most requirements have exactly one rule check (Figure 5.2). The labels are unbalanced on purpose: one app is mostly compliant (`hireright`) and one mostly non-compliant (`quickhire`). Security and transparency have the most applicable labels (Figure 5.3).

The benchmark contains deliberate *traps*:

- **for rules, indirect evidence:** logging done by a `DecisionJournal` instead of a logger, or a kill switch called `suspend_automated_ranking`;

Table 5.1: Key numbers of the data.

Policies	EU AI Act (selected parts), GDPR (selected articles), a fictional company AI policy
Clauses / requirements / checks	170 / 45 / 49; 3 requirements have no check
Applications	5: 3 high-risk (2 CV ranking, 1 credit scoring), 1 limited (chatbot), 1 minimal (spam filter)
Size per application	13–20 files, 14–26 evidence pieces
Labels	225 = 45 requirements × 5 apps; 60 not applicable; 165 applicable: 100 compliant, 14 partial, 51 non-compliant (65 gaps)

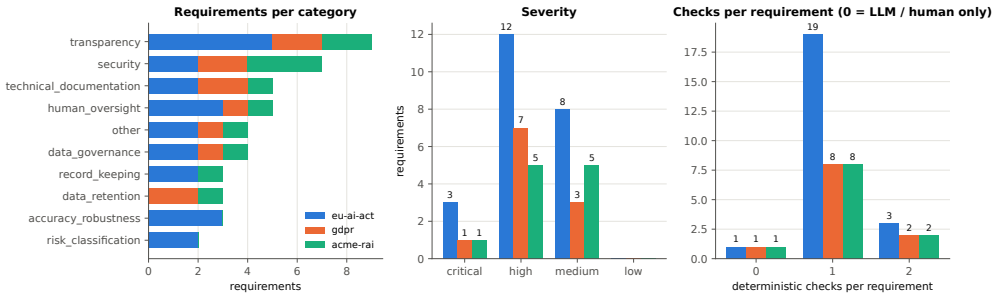


Figure 5.2: Requirements per category, severity and number of rule checks, for each policy.

- **for rules, decoys:** a commented-out `# logger.info(...)`, `override_allowed = False`, a TODO to label AI replies;
- **for LLMs:** README claims that the code does not back, and controls that exist but are not enough (a data retention period that is too long).

5.1.3 The three decision methods

Rules. Each requirement has one or more checks: a text pattern that must be present (or absent) in some files, or a file that must exist. If all checks pass, the answer is compliant; if none pass, non-compliant; if some pass, partial; if none can run, needs review. In v0 the confidence is fixed for each status (0.9 for compliant, 0.8 for non-compliant, 0.5 for partial, 0.2 for needs review). The patterns read raw text, so a commented-out line counts as real code.

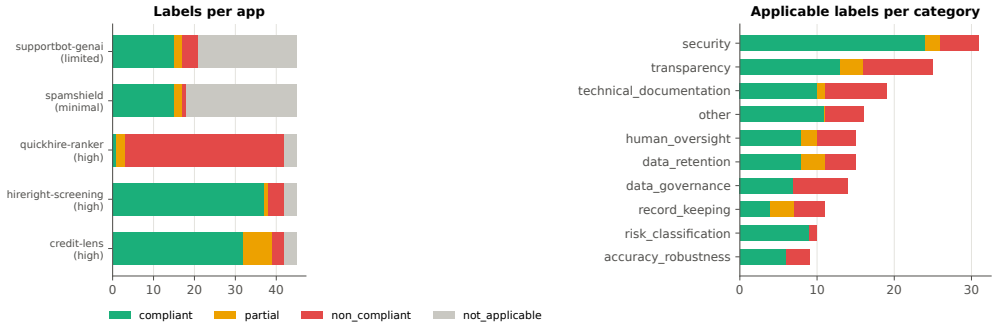


Figure 5.3: True labels per application (left) and applicable labels per category (right).

LLM judge. We use Claude Sonnet without examples or training (zero-shot), called through the Claude Code command line [2] with all tools turned off. For each requirement it receives the requirement text and at most 14 evidence pieces found by search, each with a short id (`E1`, `E2`, …). It must answer in a fixed JSON format: status, confidence, explanation, cited ids, gaps and a suggested fix. It may decide only from the pieces it sees, and is told that "absence of evidence is not compliance".

Hybrid. The rules run first. The LLM is asked only when the rules are unsure: their answer is partial or needs review, their confidence is below 0.75, or the requirement has no checks. All other rule answers are kept. If rules and LLM give opposite answers (compliant against non-compliant), the item goes to needs review; otherwise the LLM wins. Any answer that involves the LLM and has confidence below 0.6 also goes to needs review. Because the v0 rule confidences are fixed, in practice only partial and needs-review rule answers reach the LLM.

Citation check. After every answer, a small program removes cited ids that were never shown. If an LLM says compliant or partial and no valid citation is left, the answer becomes needs review.

5.1.4 Training and testing

Nothing is trained. We wrote the rules by hand from the policy text, the LLM is used zero-shot, and the two thresholds (0.75 and 0.6) were fixed before the experiments. We wrote all 225 labels before running any method, so they are a pure test set; there is no training or validation split. The main risk is that the same team wrote the rules, the prompt and the apps. To limit it, we labelled first, planted traps against our own rules, and changed nothing after seeing the v0 results.

5.1.5 Validation and comparisons

The three methods get the same requirements, evidence, search results and risk tier. Only the decision step changes, so any difference in the results comes from it. "Rules only" stands for policy-as-code, "LLM only" for an LLM judge, and the hybrid is our combination.

5.1.6 A worked example

Take requirement `AIA-REC-01` (Article 12, automatic logging) on the app `hireright-screening`. The true label is compliant: every ranking is recorded by `DecisionJournal.record(...)`. The app is high-risk, so the requirement applies.

- **Rules:** the logging pattern finds no logger, so the answer is non-compliant with confidence 0.8 (wrong).
- **Hybrid:** 0.8 is not below 0.75, so the rule answer is kept and the LLM is not asked (wrong).
- **LLM only:** the search finds `journal.py`; the judge says compliant (0.82) and cites 5 pieces (right).

The lesson: a fixed rule confidence makes a pattern that found nothing look as sure as one that found something.

5.2 Numerical validation

5.2.1 Methodology

We test three hypotheses:

- **H1:** the LLM judge finds more gaps than the rules;

- **H2:** the hybrid is more accurate than both, and cheaper than the LLM alone;
- **H3:** the rules are fooled by decoys and miss indirect evidence.

We change the decision method and measure the metrics of Chapter 3. Each method audited the five apps against all 45 requirements once, on an Apple M4 laptop (16 GB); LLM answers are cached so a run can be replayed exactly. For uncertainty we use 95% bootstrap confidence intervals (10 000 resamples of the 165 items). To compare two methods on the same items we use the McNemar test, which looks only at the items where one method is right and the other is wrong.

5.2.2 Data

The test set is the benchmark of Table 5.1. A label is *compliant* if every part of the requirement has evidence, *partial* if some parts do, and *non-compliant* if none do. Claims, TODOs and commented-out code are not evidence. *Not applicable* is used only when the risk tier excludes the requirement, so it is never a judgement call.

5.2.3 Results

Table 5.2: v0 results on the 165 applicable labels (%). Small brackets: 95% bootstrap intervals. False compl.: share of real gaps called compliant (lower is better). Calls, cost (USD) and time are for the five audits. Best value in bold.

Method	Accuracy	Macro-F1	Gap recall	False compl.	LLM calls	Cost $	s/app
Rules only	79.4 [72.7, 85.5]	72.3 [61.4, 81.6]	87.7	12.3 [4.8, 20.9]	**0**	**0.00**	**0.2**
LLM only (Sonnet)	71.5 [64.8, 78.2]	67.6 [61.0, 73.8]	**100.0**	**0.0** [0.0, 0.0]	170	4.81	94
Hybrid (Sonnet)	**86.1** [80.6, 90.9]	**77.1** [66.0, 86.1]	87.7	12.3 [4.8, 20.9]	23	0.67	25

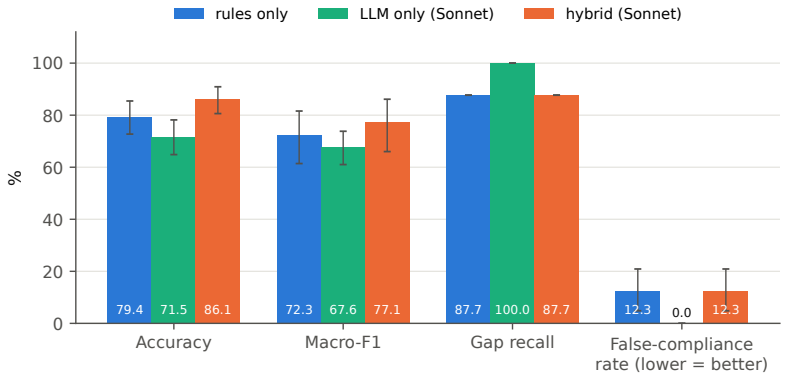


Figure 5.4: v0 results on the 165 applicable labels, with 95% bootstrap intervals (gap recall without interval).

Table 5.2 and Figure 5.4 show the results. The hybrid is the most accurate and costs 14% of the LLM-only run. It is significantly better than the rules ($p < 0.001$: it fixes 11 items and breaks none) and than the LLM ($p < 0.01$); rules against LLM is not significant ($p = 0.15$). The 18 items that the hybrid sends to the LLM reach 94.4% accuracy. The LLM alone flags all 65 gaps, while rules and hybrid miss the same 8 ($p < 0.01$). But only 60.8% of the LLM's flags are real gaps. The keyword risk tier is right for 4 of 5 apps, the LLM tier for all 5. All citations are valid for all methods.

Error analysis.

- **Rules fooled by decoys:** 4 of the 8 rule false compliances come from a commented-out logger, `override_allowed = False` and a TODO note.
- **Rules miss indirect evidence:** 10 compliant items are called non-compliant because the code uses other names (`DecisionJournal`, `suspend_automated_ranking`, "pseudonymous").
- **LLM too strict:** 41 of its 47 errors are compliant items called partial. The prompt does not say what is "enough", and cut-off evidence counts as missing.
- **Hybrid inherits rule errors:** 22 of its 23 errors are confident rule answers that never reached the LLM.

5.3 Discussion

- **H1 supported:** gap recall 100% against 87.7%, but with 42 false alarms against 16.
- **H2 partly supported:** best accuracy at 14% of the LLM cost, but the false-compliance rate stays at 12.3%, and the macro-F1 differences are within the intervals.
- **H3 supported:** decoys cause half of the rule false compliances, and indirect evidence causes 10 errors.

Strengths: rules are free, instant and precise when a requirement has a clear text pattern; the LLM reads code and prose and is not fooled by decoys; every answer points to its evidence. *Weaknesses:* the fixed rule confidence carries no information, so the hybrid cannot catch rule errors; the LLM is too strict; and the benchmark is small (the intervals are about ±5 points), synthetic and labelled by us.

6 Conclusion and future work

We defined the task of checking a repository against a policy, with a status, a confidence and cited evidence for each requirement. We built a pipeline that turns policies and repositories into an audit report, and we compared three decision methods on five hand-labelled applications. The v0 hybrid is the most accurate (86.1%) at a fraction of the LLM cost. The LLM alone is the safest: it never calls a real gap compliant. Every error comes with the evidence that caused it, so it can be found and fixed.

TODO (Seminars 6 and 7):

- **Seminar 6:** split the work among agents (policy analyst, risk classifier, evidence scout, judge, verifier, fixer); fix the v0 errors (rules that ignore comments, graded rule confidence, a scoring guide in the judge prompt, a risk heuristic that understands "not"); ablations, leave-one-app-out evaluation and significance tests against v0.
- **Seminar 7:** web dashboard, API, SARIF output and CI integration; final analysis, conclusions and presentation.

Bibliography

[1] Orlando Amaral Cejas, Muhammad Ilyas Azeem, Sallam Abualhaija, and Lionel C. Briand. NLP-based automated compliance checking of data processing agreements against GDPR. *IEEE Transactions on Software Engineering*, 49(9):4282–4303, 2023. doi: 10.1109/TSE.2023.3288901. arXiv:2209.09722.

[2] Anthropic. Claude code documentation: headless mode and command-line reference, 2025. URL https://docs.anthropic.com/en/docs/claude-code/overview.

[3] European Parliament and Council of the European Union. Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (general data protection regulation). Official

Journal of the European Union, L 119, 4 May 2016, 2016. URL http://data.europa.eu/eli/reg/2016/679/oj.

[4] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, L series, 12 July 2024, 2024. URL http://data.europa.eu/eli/reg/2024/1689/oj.

[5] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6465–6488, 2023. doi: 10.18653/v1/2023.emnlp-main.398. arXiv:2305.14627.

[6] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, et al. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2308.11462. arXiv:2308.11462.

[7] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 9459–9474, 2020. URL https://arxiv.org/abs/2005.11401. arXiv:2005.11401.

[9] Jakob Mökander, Jonas Schuett, Hannah Rose Kirk, and Luciano Floridi. Auditing large language models: a three-layered approach. *AI and Ethics*, 4(4):1085–1115, 2023. doi: 10.1007/s43681-023-00289-2. arXiv:2302.08500.

[10] OASIS. Static analysis results interchange format (SARIF) version 2.1.0. OASIS Standard, 27 March 2020, 2020. URL https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html.

[11] Open Policy Agent contributors. Open policy agent: policy-based control for cloud native environments (rego policy language). Cloud Native Computing Foundation graduated project, 2024. URL https://www.openpolicyagent.org/docs/latest/.

[12] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 2009. doi: 10.1561/1500000019.

[13] Damiano Torre, Sallam Abualhaija, Mehrdad Sabetzadeh, Lionel Briand, Katrien Baetens, Peter Goes, and Sylvie Forastier. An AI-assisted approach for checking the completeness of privacy policies against GDPR. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*, pages 136–146, 2020. doi: 10.1109/RE48521.2020.00025.

[14] Michael Veale and Frederik Zuiderveen Borgesius. Demystifying the draft EU artificial intelligence act: Analysing the good, the bad, and the unclear elements of the proposed approach. *Computer Law Review International*, 22(4):97–112, 2021. doi: 10.9785/cri-2021-220402. arXiv:2107.03721.

[15] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.05685. arXiv:2306.05685.